ES6

JavaScript Moderne

ECMAScript 2015 — Résumé complet du cours

01	**Déclaration de variables**	let, const, var
02	**Fonctions fléchées**	Syntaxe & this
03	**Template literals**	Interpolation & tagged
04	**Destructuration**	Tableaux & objets
05	**Spread & Rest**	Opérateur ...
06	**Classes**	POO & héritage
07	**Modules**	import / export
08	**Promises & async/await**	Asynchrone
09	**Itérateurs, Map & Set**	Nouvelles collections
10	**Paramètres & objets**	Raccourcis ES6

Ce document couvre toutes les fonctionnalités majeures introduites par ES6 (ECMAScript 2015) avec des explications détaillées, des exemples de code commentés et des conseils pratiques.

Chapitre 01

Déclaration de variables

ES6 introduit let et const pour remplacer var. Ils apportent la portée de bloc (block scope) et éliminent les bugs liés au hoisting involontaire.

1.1 Le problème avec var

Avant ES6, var était le seul mot-clé pour déclarer des variables. Il présente deux comportements problématiques : une portée de fonction (pas de bloc) et le hoisting, qui remonte la déclaration en haut du scope. Ces comportements sont sources de bugs difficiles à détecter.

```
// var — portée de fonction
function exemple() {
  if (true) {
    var x = 10;     // déclaré dans le if
  }
  console.log(x);  // 10 — x est accessible ici (bug !)
}

// var — hoisting
console.log(y); // undefined (pas d'erreur !)
var y = 5;
// Equivalent à : var y; console.log(y); y = 5;
```

1.2 let — la variable de bloc

let déclare une variable limitée au bloc { } dans lequel elle est définie. Elle peut être réassignée mais ne peut pas être redéclarée dans le même scope. Elle n'est pas accessible avant sa déclaration (Temporal Dead Zone).

```
let compteur = 0;
compteur = 1;          // OK — réassignation autorisée

if (true) {
  let x = 10;
  console.log(x);      // 10
}
// console.log(x);     // ReferenceError — x hors scope

// Pas de hoisting utilisable
// console.log(z);     // ReferenceError (Temporal Dead Zone)
let z = 5;
```

1.3 const — la constante de bloc

const déclare une liaison constante — la variable ne peut pas être réassignée. Attention : pour les objets et tableaux, la référence est constante mais le contenu reste mutable. C'est la déclaration à privilégier par défaut.

```
const PI = 3.14159;
// PI = 3;              // TypeError — réassignation interdite

// Objet : la référence est constante, le contenu est mutable
const user = { nom: 'Alice', age: 30 };
user.age = 31;          // OK — mutation de propriété
// user = {};           // TypeError — réassignation interdite

// Tableau : même principe
const arr = [1, 2, 3];
arr.push(4);            // OK
// arr = [];            // TypeError
```

> *Règle pratique : toujours utiliser const par défaut. Passer à let uniquement quand la valeur doit changer. Ne plus utiliser var dans du code ES6+.*

Chapitre 02

Fonctions fléchées

Syntaxe concise pour écrire des fonctions. Elles héritent du this de leur contexte englobant, ce qui résout un problème classique de JavaScript.

2.1 Syntaxes disponibles

Les fonctions fléchées offrent plusieurs niveaux de concision selon le cas d'usage.

```
// Fonction classique
function ajouter(a, b) {
  return a + b;
}

// Fléchée — parenthèses + corps + return implicite
const ajouter = (a, b) => a + b;

// Un seul paramètre — pas de parenthèses nécessaires
const double = n => n * 2;
const saluer = nom => `Bonjour, ${nom} !`;

// Aucun paramètre — parenthèses vides obligatoires
const now = () => new Date();

// Corps multi-lignes — accolades + return explicite
const clamp = (val, min, max) => {
  if (val < min) return min;
  if (val > max) return max;
  return val;
};

// Retourner un objet littéral — parenthèses obligatoires
const creerUser = (nom, age) => ({ nom, age, actif: true });
```

2.2 Comportement de this

C'est la différence la plus importante. Une fonction classique crée son propre this au moment de l'appel. Une fonction fléchée capture le this de son scope de définition (lexical this). Cela élimine le besoin de .bind(this) ou de const self = this.

```
// Problème classique avec function
class Timer {
  constructor() { this.secondes = 0; }

  demarrerAvecFunction() {
    setInterval(function() {
      this.secondes++;  // 'this' est undefined (mode strict) ou window !
    }, 1000);
  }

  // Solution avec fléchée
  demarrer() {
    setInterval(() => {
      this.secondes++;  // 'this' = l'instance Timer ✓
    }, 1000);
  }
}
```

Ne pas utiliser les fonctions fléchées comme méthodes d'objet, comme constructeurs (new), ni avec arguments (objet non disponible). Elles n'ont pas non plus de prototype.

Chapitre 03

Template literals

Chaînes de caractères délimitées par des backticks (`) permettant l'interpolation d'expressions JavaScript et les chaînes multi-lignes natives.

3.1 Interpolation et expressions

```
const prenom = 'Alice';
const age = 30;
const prix = 9.99;
const qte = 3;

// Interpolation simple
const msg = `Bonjour, ${prenom} ! Tu as ${age} ans.`;

// Expressions complexes dans ${}
const total = `Total : ${(prix * qte).toFixed(2)} EUR`;
const label = `Statut : ${age >= 18 ? 'majeur' : 'mineur'}`;
const appel = `Résultat : ${Math.max(1, 2, 3)}`;

// Multi-lignes (les sauts de ligne sont conservés)
const html = `
  <article>
    <h2>${prenom}</h2>
    <p>Age : ${age} ans</p>
  </article>
`;
```

3.2 Tagged templates (avancé)

Un template peut être précédé d'une fonction (tag) qui reçoit les parties statiques et les valeurs interpolées séparément. Utile pour la sanitisation, l'i18n, les requêtes SQL sécurisées.

```
// Tag = fonction(strings, ...valeurs)
function securiser(strings, ...valeurs) {
  return strings.reduce((acc, str, i) => {
    const val = valeurs[i] ? echapper(String(valeurs[i])) : '';
    return acc + str + val;
  }, '');
}

const input = '<script>alert("xss")</script>';
const safe = securiser`Valeur reçue : ${input}`;
// La valeur est échappée avant insertion

// Usage réel : css-in-js, graphql, sql
const query = gql`
  query GetUser($id: ID!) {
    user(id: $id) { nom email }
  }
`;
```

Les template literals supportent l'imbrication : on peut mettre un template dans un ${ } d'un autre template, mais c'est rarement lisible. Préférer des variables intermédiaires.

Chapitre 04

Destructuration

Syntaxe permettant d'extraire des valeurs de tableaux ou de propriétés d'objets en une seule instruction, avec support des alias et valeurs par défaut.

4.1 Destructuration de tableaux

```
// Extraction positionnelle
const [a, b, c] = [1, 2, 3];   // a=1, b=2, c=3

// Ignorer des éléments avec des virgules
const [, second, , quatrieme] = [10, 20, 30, 40];  // second=20

// Valeur par défaut (si undefined)
const [x = 0, y = 0] = [5];    // x=5, y=0

// Swap de variables (sans variable temporaire)
let p = 1, q = 2;
[p, q] = [q, p];               // p=2, q=1

// Récupérer le reste
const [tete, ...queue] = [1, 2, 3, 4]; // tete=1, queue=[2,3,4]

// Depuis le retour d'une fonction
const [min, max] = getRange();
```

4.2 Destructuration d'objets

```
const user = { nom: 'Alice', age: 30, ville: 'Paris' };

// Extraction par nom de propriété
const { nom, age } = user;

// Alias (renommage)
const { nom: prenom, age: annees } = user;
// prenom = 'Alice', annees = 30

// Valeur par défaut
const { role = 'visiteur', nom: n2 } = user;
// role = 'visiteur' (propriété absente)

// Reste de l'objet
const { nom: n3, ...reste } = user;
// reste = { age: 30, ville: 'Paris' }

// Objets imbriqués
const { adresse: { rue, cp } } = {
  adresse: { rue: 'Rue de Rivoli', cp: '75001' }
};

// Dans les paramètres de fonction
function afficherUser({ nom, age = 0, role = 'user' }) {
  console.log(`${nom} (${age}) — ${role}`);
}
afficherUser(user); // Alice (30) — user
```

> *La destructuration ne modifie pas l'objet source — elle crée de nouvelles liaisons. Pour les valeurs par défaut, elles ne s'appliquent que si la valeur est undefined (pas null ou 0).*

■■ Chapitre 05

Spread & Rest

L'opérateur ... a deux rôles opposés selon le contexte : spread étale un itérable, rest regroupe des valeurs en tableau.

5.1 Spread — étaler des valeurs

```
// ■■ Tableaux ■■
const a = [1, 2, 3];
const b = [4, 5, 6];

const fusion = [...a, ...b];        // [1, 2, 3, 4, 5, 6]
const copie  = [...a];              // copie superficielle
const ajout  = [0, ...a, 4];       // [0, 1, 2, 3, 4]

// Passer un tableau comme arguments
Math.max(...[3, 1, 4, 1, 5]);      // 5
console.log(...['a', 'b', 'c']);   // a b c

// ■■ Objets (ES2018) ■■
const base   = { theme: 'dark', lang: 'fr' };
const config = { ...base, lang: 'en', debug: true };
// { theme: 'dark', lang: 'en', debug: true }
// Note : lang est écrasé par la valeur suivante

// Copie superficielle d'objet
const copieUser = { ...user };

// Convertir un Set en tableau (dédupliqué)
const unique = [...new Set([1, 2, 2, 3])]; // [1, 2, 3]
```

5.2 Rest — regrouper des valeurs

```
// Dans les paramètres de fonction
function somme(premier, second, ...reste) {
  const total = reste.reduce((acc, n) => acc + n, premier + second);
  return total;
}
somme(1, 2, 3, 4, 5); // 15

// Paramètre rest seul (remplace arguments)
const log = (...args) => console.log(...args);

// Dans la destructuration de tableau
const [head, ...tail] = [10, 20, 30, 40];
// head = 10, tail = [20, 30, 40]

// Dans la destructuration d'objet
const { a: aVal, ...autres } = { a: 1, b: 2, c: 3 };
// aVal = 1, autres = { b: 2, c: 3 }
```

Le spread copie en surface (shallow copy) : les objets imbriqués restent des références partagées. Pour une copie profonde, utiliser structuredClone(obj) (natif moderne) ou JSON.parse(JSON.stringify(obj)).

Chapitre 06

Classes

Sucre syntaxique sur la chaîne de prototypes JavaScript. Les classes ES6 rendent la programmation orientée objet plus lisible et intuitive.

6.1 Déclaration et instanciation

```
class Animal {
  // Constructeur — appelé avec new Animal(...)
  constructor(nom, age) {
    this.nom = nom;
    this.age = age;
  }

  // Méthode d'instance
  presenter() {
    return `Je suis ${this.nom}, j'ai ${this.age} ans.`;
  }

  // Méthode statique — appelée sur la classe, pas l'instance
  static creer(nom, age) {
    return new Animal(nom, age);
  }
}

const chat = new Animal('Mimi', 3);
console.log(chat.presenter());   // Je suis Mimi, j'ai 3 ans.
const chien = Animal.creer('Rex', 5);
```

6.2 Héritage avec extends et super

```
class Chien extends Animal {
  constructor(nom, age, race) {
    super(nom, age);    // OBLIGATOIRE avant tout this
    this.race = race;
  }

  // Surcharge de méthode (override)
  presenter() {
    return `${super.presenter()} Je suis un ${this.race}.`;
  }

  aboyer() {
    return `${this.nom} dit : Woof !`;
  }
}

const rex = new Chien('Rex', 5, 'Labrador');
rex.presenter(); // Je suis Rex, j'ai 5 ans. Je suis un Labrador.
rex instanceof Chien;  // true
rex instanceof Animal; // true
```

6.3 Getters, setters et champs privés

```
class Compte {
  #solde = 0;           // champ privé (ES2022)
  #historique = [];

  constructor(proprietaire) {
    this.proprietaire = proprietaire;
  }

  // Getter — accès comme une propriété
  get solde() {
    return this.#solde;
  }

  // Setter — validation avant affectation
  set solde(montant) {
    if (montant < 0) throw new RangeError('Solde négatif interdit');
    this.#solde = montant;
  }

  deposer(montant) {
    this.#solde += montant;
    this.#historique.push({ type: 'dépôt', montant });
  }

  get historique() {
    return [...this.#historique]; // copie — protège l'original
  }
}

const c = new Compte('Alice');
c.deposer(100);
console.log(c.solde);  // 100
// c.#solde             // SyntaxError — accès privé interdit
```

Les classes ne sont pas hoistées : elles doivent être déclarées avant leur première utilisation, contrairement aux fonctions classiques.

Chapitre 07

Modules ES6

Système natif d'import/export pour organiser le code en fichiers indépendants. Chaque module a son propre scope — rien n'est exposé par défaut.

7.1 Export

```
// math.js

// Export nommé — autant qu'on veut par fichier
export const PI = 3.14159;
export const E  = 2.71828;

export function carre(n) {
  return n * n;
}

export const cube = n => n ** 3;

// Export d'une classe
export class Vecteur {
  constructor(x, y) { this.x = x; this.y = y; }
  longueur() { return Math.sqrt(this.x**2 + this.y**2); }
}

// Export par défaut — un seul par fichier
export default function addition(a, b) {
  return a + b;
}

// Export groupé à la fin (équivalent)
// export { PI, carre, Vecteur };
```

7.2 Import

```
// Import nommé — noms exacts
import { PI, carre, Vecteur } from './math.js';

// Import avec alias
import { carre as c2, cube as c3 } from './math.js';

// Import par défaut (nom libre)
import addition from './math.js';

// Import par défaut + nommés en une ligne
import addition, { PI, carre } from './math.js';

// Importer tout dans un namespace
import * as Math2 from './math.js';
Math2.carre(4);  // 16
Math2.default(2, 3); // 5 — l'export default

// Import dynamique (asynchrone, ES2020)
const { carre } = await import('./math.js');
```

Dans le HTML : <script type="module" src="app.js"></script>. Les modules s'exécutent en mode strict automatiquement, sont différés (defer) par défaut, et ne polluent pas le scope global.

Chapitre 08

Promises & async/await

Les Promises représentent une valeur disponible maintenant, dans le futur ou jamais. async/await (ES2017) permet d'écrire du code asynchrone avec la lisibilité du synchrone.

8.1 Créer et chaîner des Promises

```
// Créer une Promise
const fetchData = () => new Promise((resolve, reject) => {
  setTimeout(() => {
    const ok = Math.random() > 0.3;
    if (ok) resolve({ id: 1, nom: 'Alice' });
    else    reject(new Error('Connexion échouée'));
  }, 1000);
});

// Chaînage .then().catch().finally()
fetchData()
  .then(data  => { console.log('Données :', data); return data.id; })
  .then(id    => console.log('ID :', id))
  .catch(err  => console.error('Erreur :', err.message))
  .finally(()  => console.log('Requête terminée'));
```

8.2 Méthodes statiques de Promise

```
// Promise.all — toutes en parallèle (échoue si une échoue)
const [user, posts] = await Promise.all([
  fetch('/api/user').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
]);

// Promise.allSettled — toutes, succès ou échec
const resultats = await Promise.allSettled([p1, p2, p3]);
resultats.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});

// Promise.race — la plus rapide gagne
const premiere = await Promise.race([lent, rapide]);

// Promise.any — la première réussie (ES2021)
const firstSuccess = await Promise.any([p1, p2, p3]);
```

8.3 async / await

```
// async — la fonction retourne toujours une Promise
async function chargerUser(id) {
  try {
    // await suspend l'exécution jusqu'à la résolution
    const response = await fetch(`/api/users/${id}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const data = await response.json();
    return data;

  } catch (err) {
    console.error('Erreur lors du chargement :', err.message);
    throw err; // re-lancer si le caller doit gérer
  }
}

// Version fléchée async
const getData = async (url) => {
  const res = await fetch(url);
  return res.json();
};

// Parallélisme avec await
async function chargerTout() {
  const [user, config] = await Promise.all([
    getData('/api/user'),
    getData('/api/config'),
  ]);
  return { user, config };
}
```

await ne bloque que la fonction async courante, pas le thread. Pour des opérations séquentielles, enchaîner les await. Pour des opérations parallèles, utiliser Promise.all avec await.

Chapitre 09

Itérateurs, Map & Set

De nouvelles structures de données et le protocole itérable pour un parcours flexible et personnalisé des collections.

9.1 Map — dictionnaire à clés quelconques

Map est un dictionnaire où les clés peuvent être de n'importe quel type (objets, fonctions, primitives). Contrairement aux objets, l'ordre d'insertion est garanti.

```
const map = new Map();

// Clés de tout type
map.set('string', 'valeur 1');
map.set(42, 'valeur 2');
map.set(true, 'valeur 3');
const obj = {};
map.set(obj, 'valeur 4');   // un objet comme clé !

// Lecture
map.get('string');          // 'valeur 1'
map.has(42);                // true
map.size;                   // 4
map.delete(true);           // supprime

// Initialisation depuis un tableau de paires
const config = new Map([
  ['host', 'localhost'],
  ['port', 3000],
  ['debug', true],
]);

// Itération
for (const [cle, valeur] of config) {
  console.log(`${cle} => ${valeur}`);
}
[...config.keys()];         // ['host', 'port', 'debug']
[...config.values()];       // ['localhost', 3000, true]
[...config.entries()];      // [['host','localhost'], ...]
```

9.2 Set — collection de valeurs uniques

```
const set = new Set([1, 2, 2, 3, 3, 3]);
// {1, 2, 3} — doublons supprimés automatiquement

set.add(4);
set.has(2);    // true
set.delete(1);
set.size;      // 3

// Astuce : dédupliquer un tableau
const arr = [1, 1, 2, 3, 3, 4];
const unique = [...new Set(arr)]; // [1, 2, 3, 4]

// Opérations ensemblistes
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);

const union        = new Set([...a, ...b]);       // {1,2,3,4,5,6}
const intersection = new Set([...a].filter(x => b.has(x))); // {3,4}
const difference   = new Set([...a].filter(x => !b.has(x))); // {1,2}
```

9.3 for...of et le protocole itérable

```
// for...of fonctionne sur tout itérable
for (const val  of [1, 2, 3])     { }  // tableau
for (const char of 'ES6')         { }  // string
for (const val  of new Set([...])){ }  // Set
for (const [k,v] of new Map([...])){ } // Map

// Créer un objet itérable personnalisé
const range = {
  from: 1,
  to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last  = this.to;
    return {
      next() {
        if (current <= last) return { value: current++, done: false };
        return { value: undefined, done: true };
      }
    };
  }
};

for (const n of range) {
  console.log(n); // 1, 2, 3, 4, 5
}
console.log([...range]); // [1, 2, 3, 4, 5]
```

> *WeakMap et WeakSet sont des variantes dont les clés (objets) sont des références faibles — elles n'empêchent pas le garbage collector. Utiles pour associer des métadonnées à des objets sans créer de fuites mémoire.*

Chapitre 10

Paramètres & objets améliorés

ES6 enrichit la syntaxe des fonctions et des littéraux objets avec de nombreux raccourcis qui réduisent la verbosité du code.

10.1 Paramètres par défaut

```
// Avant ES6
function saluer(nom, titre) {
  nom   = nom   || 'Monde';
  titre = titre || 'M.';
  return titre + ' ' + nom;
}

// ES6 — valeurs par défaut
function saluer(nom = 'Monde', titre = 'M.') {
  return `${titre} ${nom}`;
}
saluer();             // 'M. Monde'
saluer('Alice', 'Mme'); // 'Mme Alice'

// Les expressions sont évaluées à chaque appel
function creerId(base, suffixe = Date.now()) {
  return `${base}-${suffixe}`;
}

// Référencer des paramètres précédents
function multiplier(a, b = a * 2) {
  return a * b;
}
multiplier(3);   // 3 * 6 = 18
```

10.2 Propriétés et méthodes raccourcies

```
const nom = 'Alice', age = 30;

// Shorthand properties (valeur = variable du même nom)
const user = { nom, age };
// équivaut à : { nom: nom, age: age }

// Méthodes raccourcies
const compteur = {
  valeur: 0,
  incrementer() { this.valeur++; },  // sans 'function'
  decrementer() { this.valeur--; },
  reset() { this.valeur = 0; },
};

// Getters/setters dans un littéral
const cercle = {
  _rayon: 5,
  get rayon()      { return this._rayon; },
  set rayon(r)     { if (r > 0) this._rayon = r; },
  get surface()    { return Math.PI * this._rayon ** 2; },
};
```

10.3 Clés dynamiques et Symbol

```
// Computed property names
const cle = 'couleur';
const prop = 'taille';

const style = {
  [cle]: 'rouge',          // { couleur: 'rouge' }
  [`${prop}Max`]: 100,     // { tailleMax: 100 }
  [`get${cle[0].toUpperCase() + cle.slice(1)}`]() {
    return this[cle];      // getCouleur()
  },
};

// Symbol — identifiant unique et immuable
const ID = Symbol('id');
const TYPE = Symbol('type');

const entite = {
  [ID]: 42,
  [TYPE]: 'utilisateur',
  nom: 'Alice',
};

entite[ID];             // 42
// Les Symbols n'apparaissent pas dans for...in ni JSON.stringify
Object.getOwnPropertySymbols(entite); // [Symbol(id), Symbol(type)]

// Méthodes utiles sur Object
Object.keys(user);           // ['nom', 'age']
Object.values(user);         // ['Alice', 30]
Object.entries(user);        // [['nom','Alice'], ['age',30]]
Object.assign({}, a, b);     // fusion (remplacé par le spread)
Object.freeze(user);         // rend immuable
```

Préférer le spread { ...obj } à Object.assign({}, obj) : plus lisible, même comportement pour la copie superficielle. Object.assign modifie le premier argument — attention aux effets de bord.

Récapitulatif rapide

Fonctionnalité	Syntaxe clé	À retenir
let / const	let x = 1 const Y = 2	const par défaut, let si mutable
Fléchée	const f = x => x*2	this lexical, pas de prototype
Template literal	\`Hello ${name}\`	backticks, ${} pour les expressions
Destructuration	const {a} = obj	alias, valeurs par défaut, rest
Spread	[...a, ...b] { ...o }	copie superficielle uniquement
Rest	(...args) =>	dernier paramètre, remplace arguments
Classes	class A extends B	super() obligatoire, pas de hoisting
Modules	import/export	portée propre, mode strict auto
Promises	.then().catch()	Promise.all pour le parallélisme
async/await	async fn / await p	syntaxe synchrone, toujours try/catch
Map	new Map()	clés de tout type, ordre garanti
Set	new Set()	valeurs uniques, déduplique un tableau
for...of	for (x of iter)	tout itérable : tableau, string, Map…

Bonne pratique générale : écrire du code ES6+ en mode strict, utiliser const et let, préférer les fonctions fléchées pour les callbacks, les classes pour la POO, les modules pour l'organisation, et async/await pour l'asynchrone.